Chapter 6: Missing Values

by Lorraine Gaudio

Version February 18, 2026

BOISE STATE UNIVERSITY

Contents

1. Overview

In this chapter, you'll build the core R habits that prevent your analysis from being quietly wrong. You'll learn how built-in functions summarize vectors, how to read help files to tell what a function requires versus what it assumes by default, and how those defaults directly affect results. Then you'll focus on missing data—what NA means, how missing values "propagate" through calculations, how to detect and locate missing values, and how to make deliberate choices about what to do next (remove missing values or use simple imputation). You'll also practice nesting functions to solve multi-step problems efficiently while still keeping your code understandable. This fits where you are in the course because you are moving from "R runs" to "R is correct": you're learning to verify assumptions, interpret defaults, and handle messy data the way real researchers must in order to produce results that others can trust (CLO1), troubleshoot systematically (CLO2), and document your reasoning and resources transparently (CLO3).

In Chapter Six you will learn how to:

- Summarize vectors with built in, *vectorized* functions.
- Read help files to discover arguments and defaults.
- Nest functions to build compact pipelines.
- Handle missing values with na.rm.

2. Open an R Notebook

Before you follow along with Chapter 6, set yourself up the same way you did in Chapter 5. Your work will to be readable and reproducible. Please open a new R Notebook in your course folder named chapter6_notes.Rmd and add a date to the YAML header using either method. Use this document to take notes and practice the chapter content.

1. Open RStudio → File → New File → R Notebook.
2. Save it in your course folder as: chapter6_notes.Rmd.
3. In your YAML header at the top:

- Update the title to match this chapter.
- Include a date (hard-coded or dynamic, your choice).

4. Use headings as you work. At minimum, create section headings that match the chapter structure (e.g., # Special Values, # Built-in Functions, # Help Files, # Missing Values Workflow, # Nesting). This creates a readable outline.
5. Memo as you go.

- Outside chunks: write normal text.
- Inside chunks: use # for memo comments (otherwise R tries to run the text).
- If you hit an error: keep the broken line commented out and write a one-sentence note about what you changed to fix it.

Bottom line: You are practicing Chapter 6 content inside a Chapter 5-style report.

3. Special Values

R has several **special values** that represent non-standard data states. Two common ones are NA and NaN.

- **NA** ("Not Available") signals *missing* information: the value simply was not recorded.
- **NaN** ("Not a Number") signals an *undefined* numeric result, such as 0 divided by 0.

```
#  Type this in your R Notebook
0 / 0      # Produces NaN – undefined arithmetic

NA         # Built-in constant representing a missing value
```

 R prints NA and NaN in the Console so you can spot them quickly. Even though they look similar, treat them differently: NA means "value absent"; NaN means "math error".

We will mostly focus on NA in this chapter, but keep in mind that NaN is another special value you may encounter when working with numeric data.

4. Built-in Functions

Many built-in summary functions take a whole vector as input and return one summary value (or a small set of values). These are summary (reduction) functions that take many values and return one value.

4.1 Function Objects vs. Function Calls

A **function object** is the stored code that performs a task. For example, mean is a function object that calculates the average of a numeric vector. The stored code sums the values and divides by the number of values.

```
# ▢ Type this in your R Notebook
mean
```

```
## function (x, ...)
## UseMethod("mean")
## <bytecode: 0x000001a708b3d230>
## <environment: namespace:base>
```

▢ Don't worry if what prints looks cryptic. This is proof that mean is stored code you can inspect.

A **function call** runs the stored code. To call a function, you type the function name followed by parentheses (). Inside the parentheses, you must provide any **required arguments**. For example, to calculate the mean of a numeric vector, you call the mean() function and provide the vector as an argument. The vector is a **required argument** because the function needs it to perform the calculation.

▢ The SYNTAX (basic):

mean(x)

where x is the numeric vector you want to summarize.

▢ Calculate the mean of the cursed sequence.

```
# ▢ Type this in your R Notebook
cursed_seq <- c(4, 8, 15, 16, 23, 42)

mean(cursed_seq)
```

```
## [1] 18
```

 The mean() function calculates the average of the numbers in the cursed_seq vector by summing them up and dividing by the count of numbers.

5. Help Files

Nobody memorizes every argument. You look them up. ?function_name opens the help file. The **files pane** (usually bottom-right) contains several tabs, including the **Help tab**, which displays the documentation for R functions.

5.1 No Required Arguments

Some functions do not have a required argument. For example, the Sys.Date() function returns the current date without needing any input.

 Call the Sys.Date() function to get the current date from your computer's operating system.

```
# Type this in your R Notebook
Sys.Date()
```

```
## [1] "2026-02-18"
```

 The Sys.Date() function retrieves the current date from the system clock and returns it as a Date object.

 Open the help file for the Sys.Date() function.

```
# Type this in your R Notebook
?Sys.Date
```

 The Sys.Date help page opens in the **help tab** in the **files pane**. Note that Sys.Date() has no required arguments.

Scroll through the help file to find information about the function's usage, arguments, and examples.

- **Usage**: shows the function signature and defaults.
- **Arguments**: what each argument does.
- **Examples**: copy/run examples to learn patterns.

5.2 Required Arguments

 Open the help file for the mean() function. Scroll through the help file to find information about the function's usage, arguments, and examples.

- **Usage**: shows the function signature and defaults.
- **Arguments**: what each argument does.
- **Examples**: copy/run examples to learn patterns.

```
#  Type this in your R Notebook
?mean
help(mean)
```

 The mean **help page** opens in the **help tab** in the **files pane**. mean.default(x, trim = 0, na.rm = FALSE, …) shows that x is a required argument, while trim and na.rm have default values.

5.3 Default Arguments

Default arguments are inputs that a function uses in situations when you did not provide that argument. For example, the trim argument in the mean() function allows you to exclude a certain percentage of the lowest and highest values before calculating the mean.

The SYNTAX (with one default argument):

mean(x, trim = 0)

 Calculate the trimmed mean of the cursed sequence, removing 20% of the lowest and highest values before computing the mean.

```
#  Type this in your R Notebook
mean(cursed_seq, trim = 0.2)  # drop 20% from each end first
```

```
## [1] 15.5
```

 The mean() function first removes 20% of the lowest and highest values from the cursed_seq vector, then calculates the mean of the remaining values. With 6 values and trim = 0.2, R trims 4 and 42, then averages 8, 15, 16, and 23.

How can you tell which arguments are required?

The main clue that x is required is that it has no = sign or default value next to it in the usage section. The other arguments have = signs, showing the default values that R will use if you don't provide your own.

Some functions use ... (dot-dot-dot). That means "additional optional arguments may be allowed," not required ones. For now, ignore ... until you are more advanced.

- no = → required
- = → default/optional

6. Missing Values Workflow

The na.rm argument in the mean() function has a default value of FALSE, meaning that if you don't specify it, the function will include NA values in its calculation.

In R, **missing values** are represented by the **special value** symbol NA (which stands for "Not Available"). Missing values can occur in datasets for various reasons, such as data entry errors, non-responses in surveys, or data corruption. R *will* do the operation, but the result becomes NA because the value is unknown (e.g., 5 + NA returns NA, not an error). This is called **propagation**.

 The SYNTAX (with na.rm default argument):

mean(x, na.rm = FALSE)

where na.rm is a logical argument that tells the function whether to remove NA values before calculating the mean.

 Add NA value(s) to the cursed sequence and calculate the mean while handling missing values.

```
# Type this in your R Notebook
cursed_seq <- c(4, 8, 15, 16, 23, 42, NA)  # includes a missing value
# cursed_seq <- c(cursed_seq, NA, NA)    # uncomment to add more missing values

mean(cursed_seq)                    # default is na.rm = FALSE → returns NA

mean(cursed_seq, na.rm = TRUE)        # Change default to TRUE to ignore missing values
```

 The first mean() call returns NA because the default behavior includes missing values in the calculation. The second call uses na.rm = TRUE to ignore the NA values, allowing the function to compute the mean of the remaining numbers.

In many summary functions, any NA in the vector causes the result to be NA unless you tell R to remove missing values first.

Function	What it does	Example
sum(), prod()	Sum/product	sum(x, na.rm = TRUE)
mean(), median()	Central tendency	mean(x, na.rm = TRUE)
min(), max(), range()	Extrema/range	range(x, na.rm = TRUE)
sd(), var()	Spread	sd(x, na.rm = TRUE)

6.1 Detecting Missing Values

A quick yes/no check prevents surprises later in your workflow.

- anyNA(x) returns TRUE if *any* element in x is NA or NaN.

 The SYNTAX (basic):

anyNA(x)

where x is the vector you want to check for missing values.

 Use anyNA() to check if there are any missing values in a vector.

```
#  Type this in your R Notebook
Vector_NA <- c(3, 7, NA, 12)  # Our sample data – one value is missing

anyNA(Vector_NA)              # TRUE means "Something is missing!"
```

 If was my data and I saw FALSE, I could relax; the vector has no gaps. Seeing TRUE tells me to investigate further.

6.2 Locating Missing Values

There are two useful functions to find the exact positions of missing values in a vector:

- is.na(x) returns a logical vector: TRUE wherever x is NA or NaN.
- which(logical_vec) converts TRUE positions into numeric indices (handy for slicing or replacing values).

 Use is.na() and which() to locate missing values in Vector_NA.

```
#  Type this in your R Notebook
is.na(Vector_NA)        # Notice that it is TRUE at position 3
```

 The is.na() function creates a logical vector indicating which elements are missing. The third element is TRUE, indicating that the third element in Vector_NA is NA.

Vectors can be quite long, so it's often more useful to get the numeric indices of missing values. Use which() to extract those indices.

```
#  Type this in your R Notebook
logical_NA <- is.na(Vector_NA)
which(logical_NA)  # Returns index 3 directly
```

 The is.na() function creates a logical vector logical_NA indicating which elements are missing, while the which() function extracts the *indices* of those missing values (see **Positional Indexing**).

7. Nesting Function Calls

Nesting function calls is where you put one function call inside another, so the output of the inner function becomes an argument to the outer function. This works because functions return values, and R can immediately pass that returned value into the next function call.

Nesting is the simplest form of function composition. Building a multi-step transformation by combining small, single-purpose functions.

 Use sample() to randomly select 5 numbers from the vector travel_jumps, then calculate the mean of those sampled numbers by nesting the sample() function inside the mean() function.

```
#  Type this in your R Notebook
# Non-nesting version
travel_jumps <- seq(from = 1887, to = 2052, by = 33)
travel_jumps_sample <- sample(travel_jumps, size = 5, replace = TRUE)
mean(travel_jumps_sample)

# Nesting version
mean(sample(travel_jumps, size = 5, replace = TRUE))  # sample then average

# Nesting version without creating the travel_jumps object
mean(sample(seq(from = 1887, to = 2052, by = 33), size = 5, replace = TRUE))
```

 The first lines of code show multiple steps, each with a single purpose: create the sequence, sample the sequence, then find the mean. The Nesting version shows that the sequence is first sampled, and then the sample mean is calculated. The final Nesting version shows that the sequence is first calculated, then sampled, and finally, the sample mean is computed.

Real work is rarely one step. Nesting is the base-R way to express that workflow without writing a ton of intermediate objects. However, nesting creates parentheses-heavy code that's harder to read and debug. In later chapters, we will learn how to pipe multi-step workflows.

7.1 Locating Missing Values with Nesting

 Create a large vector with some missing values using sample() and c().

```
#  Type this in your R Notebook
set.seed(100)
LargeVec <- sample(c(1:10, rep(NA, 10)))
print(LargeVec)
```

 We used the c() function in Chapter 2. Here we have nested c() inside of sample(). We also used the rep() function, which is covered in more detail in a later chapter.

 Use which() and is.na() to find the positions of missing values in a large vector by nesting the is.na() function inside the which() function.

```
# Type this in your R Notebook
which(is.na(LargeVec))                    # Where are the gaps?
```

 Theis.na() function creates a *logical* vector indicating which elements are missing, and the which() function determines the indices of those TRUE values. The numbers are the numeric indices (position) of the NA values.

7.2 Summarizing "Missingness"

As a researcher, knowing *how much* data is missing guides your next step.

- A single NA might be harmless.
- 50% missing will bias results and needs attention.

 Use sum() and mean() with is.na() to summarize missing values in a large vector.

```
# Type this in your R Notebook
sum(is.na(LargeVec))

mean(is.na(LargeVec))
```

 The sum(is.na(LargeVec)) expression counts the total number of missing values in LargeVec by summing the TRUE values from is.na(). The mean(is.na(LargeVec)) expression calculates the proportion of missing values by taking the mean of the logical vector returned by is.na(), where TRUE is treated as 1 and FALSE as 0. Fifty percent missing values yield a mean of 0.5.

7.3 Imputation

Now that you can *detect* and *locate* gaps, you have to decide what to do about them. There is no universal "best" choice—each option makes assumptions. If you don't know why values are missing, be cautious.

- **Option 1**: Remove missing values (complete-case analysis). Simple and common, but you may drop a lot of data (and potentially bias results).
- **Option 2**: Replace missing values with a summary value (simple imputation). Keeps the vector length the same, but it can distort the distribution (especially the variance).

In this chapter we're practicing on a single vector, so removing missing values often looks "safer." In real analysis, though, values live in rows of a dataset (one row = one person/observation) with multiple columns. If you remove rows because one column is missing, you also throw away valid information in other columns. That can:

- shrink your sample size (less stable results),
- change which groups are represented (bias),
- break analyses that require the full row structure (plots, models, group summaries).

 Example: Imagine a class dataset with student_id, major, exam1, and exam2. If exam2 is missing for some students, removing those students from the analysis (rows) means we'd also lose their exam1 scores and their major. If our goal is "average Exam 1 by major," dropping rows because Exam 2 is missing can silently change the story. This is especially true if missing Exam 2 happens more in one major than another.

The bottom line is that there is no one-size-fits-all solution. If you only need a quick summary of one variable, removing NAs is often reasonable. If you need to keep rows aligned across variables (or you're comparing groups), you may choose imputation. What ever you choose, you will need to document that choice and why. This is one reason why writing memos around you analysis is so important.

Next, you'll practice both strategies on a vector so you can see the trade-off clearly. Later, you'll apply the same decision logic to multi-column datasets.

7.3.1 Option 1

Remove missing values

 Use indexing with !is.na() to keep only the observed (non-missing) values in LargeVec.

```
#  Type this in your R Notebook
clean_vec <- LargeVec[!is.na(LargeVec)]   # Keeps only observed values
clean_vec
```

 is.na(LargeVec) returns a logical vector (TRUE for missing, FALSE for present). The ! flips it (so TRUE means "keep this value"). The brackets [...] then use that logical vector to select only the elements you want.

7.3.2 Option 2:

Impute missing values (replace NA with the mean)

Impute means "fill in missing values." A common simple imputation method is to replace missing values with the mean of the observed values.

 Make a copy of LargeVec, then replace only the missing values with the mean of the observed values.

```
# Type this in your R Notebook
imputed <- LargeVec     # Make a copy to preserve original
```

 The reason we make a copy is that we want to keep the original vector with its missing values intact for comparison. This way, we can see the effect of imputation without losing the original data.

 Tip: Once we make a copy, we use that copy name everywhere in the imputation line. This new version of the vector is called imputed, and it is the version we will use next. imputed is the vector's *name*. The name is descriptive of what the difference will be between it and the original LargeVec once we have replace only the missing values with the mean of the observed values. It is good practice to name objects in meaningful ways.

We're doing three actions on the same object (imputed):

(1) identify missing positions, (2) compute a replacement value, and (3) write the replacements back.

```
# Type this in your R Notebook
imputed[is.na(imputed)] <- mean(imputed, na.rm = TRUE)
imputed
```

 is.na(imputed) identifies exactly where the gaps are, and imputed[...] <- ... replaces only those positions. The na.rm = TRUE argument is required here. Without it, mean(imputed) would return NA because the input contains missing values.

The object we are editing is on the left side: **imputed[**is.na(imputed)**]**.

The object we are using to find the missing positions is inside brackets: **is.na(imputed)**.

 You might think, "The missing values came from scores, so I should check scores." But the rule is to use the same object we're editing, or we risk indexing the wrong thing later.

The object we are summarizing to compute the replacement value (mean): **mean(imputed, na.rm = TRUE)**

 You might think, "mean of observed values" means "mean of the original." But imputed currently equals scores, so the mean is identical either way at that moment. As soon as you start changing imputed, mixing names can cause logic bugs. You might replace values in one object using positions or summaries computed from a different object.

In summary, wherever a element in imputed is missing, we replace those spots with the mean of imputed while ignoring missing values so it can calculate the mean.

 Confirm your result by comparing sum(is.na(LargeVec)) versus sum(is.na(imputed)).

```
# 
sum(is.na(LargeVec))      # Original vector – should print number 10
sum(is.na(imputed))       # Imputed vector – should print 0
```

 The first sum(is.na(LargeVec)) call counts the number of missing values in the original vector, which should be greater than 1. The second sum(is.na(imputed)) call counts the missing values in the imputed vector, which should be 0, confirming that all missing values have been replaced.

8. Summary

In Chapter 6, you learned that R has special values for non-standard situations, especially NA (missing information) and NaN (undefined arithmetic), and you practiced built-in summary functions that reduce a vector to a single value. You also learned how to read R help files to determine what arguments are required versus optional defaults, using mean() as an example (including trim and na.rm). You built a practical missing-values workflow: detect missingness (anyNA()), locate missing values (is.na() and which()), summarize how much is missing (sum(is.na(x)) and mean(is.na(x))), and handle missing data by either removing missing values or imputing them with a summary value. Finally, you practiced nesting function calls to combine steps into one expression and discussed the trade off between compact code and readability.

9. Chapter Terms

Arguments: Inputs that you provide to functions to customize their behavior. Arguments are specified within the parentheses of a function call and can modify how the function operates or what data it processes.

Default Arguments: Inputs that already have a preset value in the function definition and are used automatically when you don't provide your own (e.g., mean(..., na.rm = FALSE) by default).

Function Call: The act of running a function by writing its name followed by parentheses, optionally supplying arguments inside (e.g., mean(x)).

Function Object: A function stored as an R object (functions are "first-class" in R, meaning you can assign them to names, pass them to other functions, and inspect them).

Help Page: Documentation for R functions and packages that provides information on usage, arguments, and examples. Accessed using ?function_name or help(function_name).

Help Tab: Located in the Files/Plots/Packages/Help pane, this tab provides access to R documentation, package vignettes, and search functionality to find help topics related to R functions and packages.

Impute: To replace missing or unusable values (e.g., NA) with substituted values—often a plausible estimate based on the observed data (such as mean/median imputation or model-based predictions)—so the dataset can be analyzed without dropping incomplete cases. Because imputed values are estimates, you should document what method you used and why.

Missing Values: Data values that are unknown or not recorded; in R they are represented by NA ("Not Available"). NA is a special missing-value indicator (with typed versions like NA_integer_, NA_real_, and NA_character_), and many calculations will return NA if any missing values are present unless you explicitly remove them (often via na.rm = TRUE, which defaults to FALSE in functions like mean())

NA: A special constant that marks a missing value ("Not Available")—meaning the data point is unknown or not recorded. R also provides typed missing values like NA_integer_, NA_real_, NA_character_, and NA_complex_ to match the underlying data type.

NaN: A special numeric constant ("Not a Number") that represents an undefined arithmetic result in floating-point math (for example, 0/0 or other invalid operations). NaN is non-finite (not a regular number).

Nesting Function Calls: Writing one function call inside another so the inner function runs first and its returned value becomes an argument to the outer function (for example, mean(sample(x, 5))). This works because a function call can appear anywhere an expression is allowed, and functions return a value that can be passed directly into the next call.

Positional Indexing: Indexing a vector using numeric positions (for example, x[1] for the first element or x[c(1, 3)] for the first and third). Positions are based on the current order of the vector.

Propagation: R's default behavior where an unknown value "spreads" through a computation. If an expression includes NA, the result will usually be NA (e.g., 5 + NA returns NA) because the outcome can't be determined without the missing information. This is often called **NA propagation**, and it's why many summary functions return NA unless you explicitly remove missing values (e.g., na.rm = TRUE).

Required Arguments: Inputs a function needs because they have no default value in the function definition; you must supply them in the call (or the function can't do its job). **Special Values** (NA, NaN, Inf, -Inf, NULL): Reserved constants in R that represent non-standard values: NA for missing data, NaN for undefined numeric results, Inf/-Inf for positive/negative infinity, and NULL for "no value" (an empty/null object). In practice, use is.na() to detect NA and NaN, is.nan() to detect NaN specifically (not NA), is.infinite()/is.finite() to handle infinities and other non-finite values, and is.null() to test for NULL.

10. ☒ Practice Space

Fill in the blanks to complete the code.

Before you begin, make sure you have created a new R Notebook in the Source pane and saved it as chapter6_practice.Rmd in your course folder.

Remember to document your tenacity on a memo if you encounter any errors while completing the tasks.

R Notebooks can look "fine" even when they are not reproducible because your Environment may still contain objects from earlier work. Start with a clean environment and run everything in order.

10.1 Task 1

☐ **Help-file scavenger hunt** (required vs default)

1. ☐ Use ?sample to learn about the sample() function.
2. In one sentence, explain how you can tell required vs default arguments from the **Usage** line.
3. Write the Usage lines exactly as they appear (copy/paste is fine), then label:

- TWO required arguments
- TWO default arguments for sample() exactly as they appear in the Usage section. How you can tell which arguments are required?

☐ *Your explanation here*

10.2 Task 2

☐ **Build a random data set**

☐ Create scores as **28 values**. That is **25** random integers from **60–100** + 3 NAs.

1. Use seq() to create 60 through 100. Store as pool.
2. Use sample() to draw 25 values (*with replacement*)
3. Add three NA to the end of scores_numbers using c(). Store as scores.
4. Verify that there are 28 elements in scores.

```
# ☐ Type this in your R Notebook, Fill in the blanks
set.seed(1)

pool <- ____                    # use seq() to create 60 through 100
scores_numbers <- ____          # use sample() to draw 25 values (with replacement)
scores <- c(____, NA, NA, NA)   # add 3 `NA` to the end of `scores_numbers` using `c()`

# ☐ Check the length of scores
_____(scores)
```

☐ *Memo: articulate your learning process, connect ideas, or discuss challenges.*

10.3 Task 3

☐ **Handle missing data**

☐ Using scores from Task 2:

1. Count missing values. Store as n_missing.
2. Compute the mean with defaults. Store as average_raw.
3. Compute the mean excluding NAs and store it as average_score.
4. Explain why average_raw behaves the way it does (use the word "default").

```
n_missing <- ____               # hint: is.na() + sum()
```

```
# ☐ Check the number of missing values
____        # should print 3
```

```
# ☐ Type this in your R Notebook, Fill in the blanks
average_raw <- mean(____)       # what happens?
```

```
# ☐ Type this in your R Notebook, Fill in the blanks
average_scores <- mean(____, ____)
# ☐ print the mean
```

In one sentence, explain why average_raw behaves the way it does (use the word "default").

☐ *Your explanation here*

☐ *Memo: articulate your learning process, connect ideas, or discuss challenges.*

10.4 Task 4

Locate missing values

Using scores from Task 2:

1. Use is.na() to create a logical vector showing which elements of scores are missing; store it as scores_is_missing.
2. Nest is.na() inside which() to find the positions (indices) of the missing values; store as missing_positions.
3. Verify your result

```
# Type this in your R Notebook, Fill in the blanks
scores_is_missing <- is.na(____)

missing_positions <- which(____(____))
```

```
# Check
print(____)       # should print a logical vector
print(____)       # should print indices of missing values
length(____)      # should print 3
```

Memo: articulate your learning process, connect ideas, or discuss challenges.

10.5 Task 5

Nesting functions

Compute the mean of 5 random draws from scores in one line, and make sure it still works even if the draw includes NA.

```
# Type this in your R Notebook, Fill in the blanks
set.seed(21)
mean_of_five <- ____
# Check
mean_of_five
```

Memo: articulate your learning process, connect ideas, or discuss challenges.

10.6 Task 6

Quick time-stamp

Use Sys.Date() and Sys.time() to create two separate objects, today_date and now_time.

```
# ☐ Type this in your R Notebook, Fill in the blanks
today_date <- _____
now_time <- _____
```

☐ *Memo: articulate your learning process, connect ideas, or discuss challenges.*

10.7 Task 7

☐ **Handling Missing Values**

☐ Using scores from Task 2:

Part A — Remove NA values ("complete-case" for a vector)

1. Create scores_clean by keeping only the observed (non-missing) values in scores.
2. Verify that scores_clean has 3 fewer values than scores.

```
# ☐ Type this in your R Notebook, Fill in the blanks

scores_clean <- scores[!_____(_____)]

# ☐ Checks

length(scores)        # should print 28
length(scores_clean)  # should print 25
```

☐ In one sentence, explain what the code that creates scores_clean does (mention indexing + !).

☐ *Your explanation here*

Part B — Impute: replace NA with the mean of present values

1. Make a copy of scores named scores_imputed (so you don't overwrite the original).
2. Replace the missing values in scores_imputed with the mean of the observed values.
3. Verify there are **0** missing values after imputation.

```
# ☐ Type this in your R Notebook, Fill in the blanks
scores_imputed <- _____

scores_imputed[____(_)] <- mean(_____, na.rm = _____)
```

```
# ☐ Checks

sum(is.na(scores_imputed))    # should print 0
mean(scores_imputed)          # should print a number (not NA)
```

☐ *Memo: articulate your learning process, connect ideas, or discuss challenges.*

10.8 Task 8

 Reflection

In one sentence, explain the difference between a *function object* (e.g., mean) and a *function call* (e.g., mean()).

 Your explanation here

10.9 Task 9

 Step 1: Sweep your Environment (start clean).
Click the **broom icon** in the Environment pane to clear objects.

 Step 2: Run everything top-to-bottom.
Use **Run → Run All** to execute every chunk in order.

 Step 3: Save and submit.
After your notebook runs without errors, save your notebook (.Rmd) and knit to HTML (.HTML) for submission to Canvas, as directed.

 Tip: After you knit, look in your course folder for two files with the same name. One file ends in .Rmd and one ends in .html. Upload the .html file. You can check which is which because one opens in R and the other usually opens in a web browser.